UNED



Departamento de Sistemas de Comunicación y Control

Tema 1: Fundamentos de bases de datos NoSQL

Agustín C. Caminero Herráez
accaminero@scc.uned.es // @accaminero

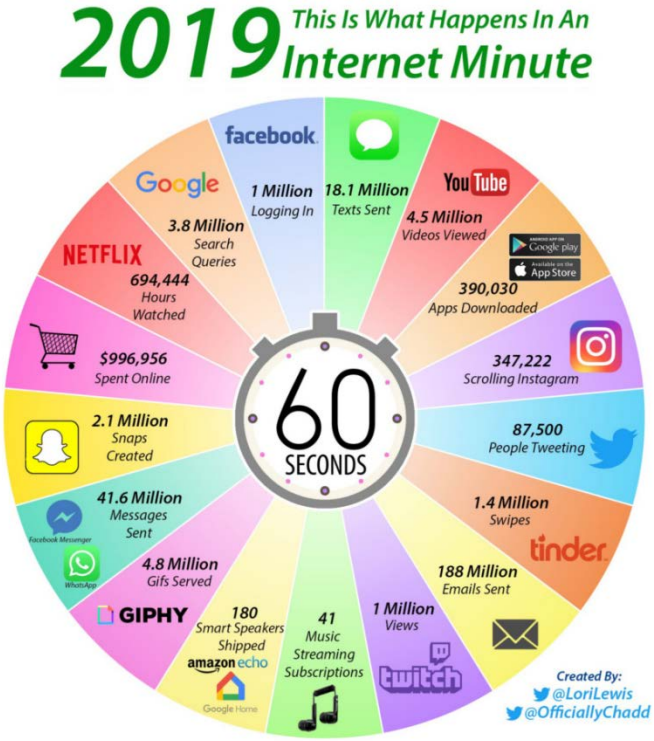
CONTENIDOS DEL TEMA

1. ¿Qué es NoSQL?
2. Teorema de CAP.
3. Modelos de datos.
4. Modelos de distribución.
5. Indexación.

1. ¿QUÉ ES NOSQL?

1. ¿QUÉ ES NOSQL?

- Son tecnologías de base de datos que pretenden abordar el problema de la **escalabilidad** en bases de datos relacionales, causada por las características de los datos masivos:
  - Volumen.
  - Velocidad.
  - Variedad.
- Las bases de datos relacionales presentan cuellos de botella:
  - Control de concurrencia.
  - Protocolos de transacciones distribuidas.



1. ¿QUÉ ES NOSQL?

- Las bases de datos relacionales son **ACID**:
  - **Atomicidad:** Indica que las transacciones de la base de datos o bien se ejecutan por completo, o bien no se ejecutan en absoluto. No es posible que una transacción se ejecute parcialmente.
  - **Consistencia:** Los datos almacenados no pueden violar las restricciones de integridad de la base de datos.
  - **Aislamiento** (**Isolation**): Múltiples transacciones ejecutándose concurrentemente no interfieren entre ellas.
  - **Durabilidad:** Las transacciones finalizadas persistirán incluso en el caso de fallos del sistema.

1. ¿QUÉ ES NOSQL?

- Las bases de datos NoSQL son **BASE** (Basically Available Soft-state Eventually consistency).
  - La base de datos está ***básicamente disponible*** la mayor parte del tiempo, es decir, aunque algunos nodos fallen en un momento determinado, la base de datos seguirá funcionando.
  - ***Soft-state*** significa que el estado del sistema no se puede garantizar, es decir, que el mismo dato puede tener múltiples valores en diferentes partes del sistema.
  - ***Consistencia eventual*** implica que aunque la base de datos no esté en el mismo estado todo el tiempo, pasado el suficiente tiempo, llegará a un estado consistente.

La consistencia es la clave.

- Relacionado con el **teorema de CAP**.

1. ¿QUÉ ES NOSQL?

NoSQL	Base de datos relacional
Almacenan datos de varias formas: key-value, documento, basado en columnas o en grafos	Almacena datos en forma tabular (filas y columnas)
Esquema dinámico	Esquema predefinido
Escalable horizontalmente	Escalable verticalmente
Soportan las operaciones CRUD básicas: Trasladan la complejidad a la aplicación. No soportan joins.	SQL soporta consultas complejas y anidadas
Son BASE	Son ACID
Se utilizan en servicios Web2.0 (redes sociales, blogs, etc.), aplicaciones IoT, almacenamiento de perfiles sociales…	Se utilizan en sistemas transaccionales (banca…),

2. TEOREMA DE CAP

2. TEOREMA DE CAP

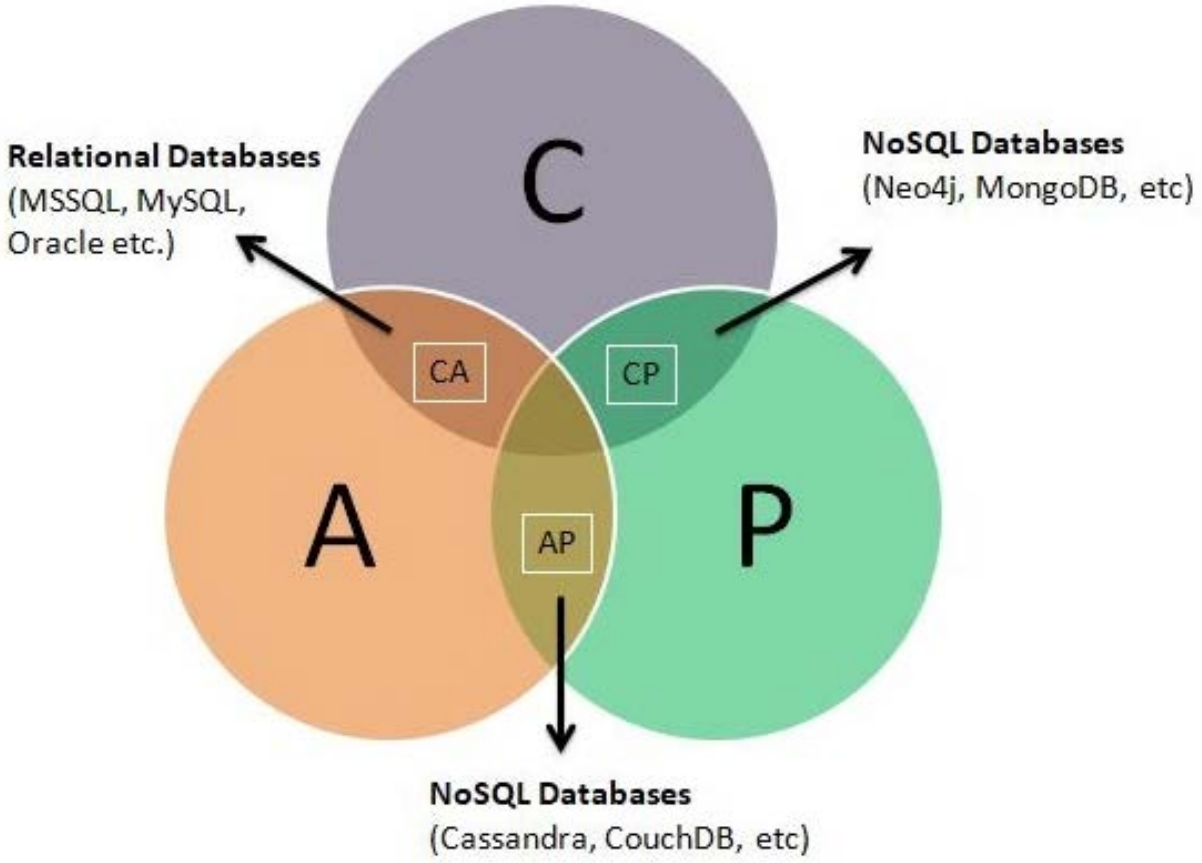
- **Teorema de CAP**: Un sistema distribuido debe idealmente proporcionar las siguientes características:
  - **Consistencia:** Cada petición de lectura sobre un dato debe devolver el resultado de la última escritura que haya tenido lugar sobre ese dato con éxito.
  - **Disponibilidad (Availability):** Independientemente del estado de cada nodo, el sistema siempre responderá a las consultas.
  - **Tolerancia al particionamiento (Partition tolerance):** El sistema continua respondiendo a las consultas aunque haya conjuntos de nodos incomunicados entre ellos (fallo del nodo, fallo de red).

2. TEOREMA DE CAP

- **Teorema de CAP**: Un sistema distribuido debe idealmente proporcionar las [illegible]ntes características:
  - **Consistencia**: [illegible]e lectu[illegible]debe devolver el resultado de la u[illegible]obre ese dato con éxito.
  - **Disponibilidad (Avail[illegible]**: [illegible]mente del estado de cada nodo, el sistema si[illegible]
  - **Tolerancia a[illegible]nto (Pa[illegible]ce):** El sistema continua respon[illegible]s consultas aunq[illegible]njuntos de nodos incomunicados (fa[illegible]el nodo, fallo de red).

IMPOSIBLE LAS 3

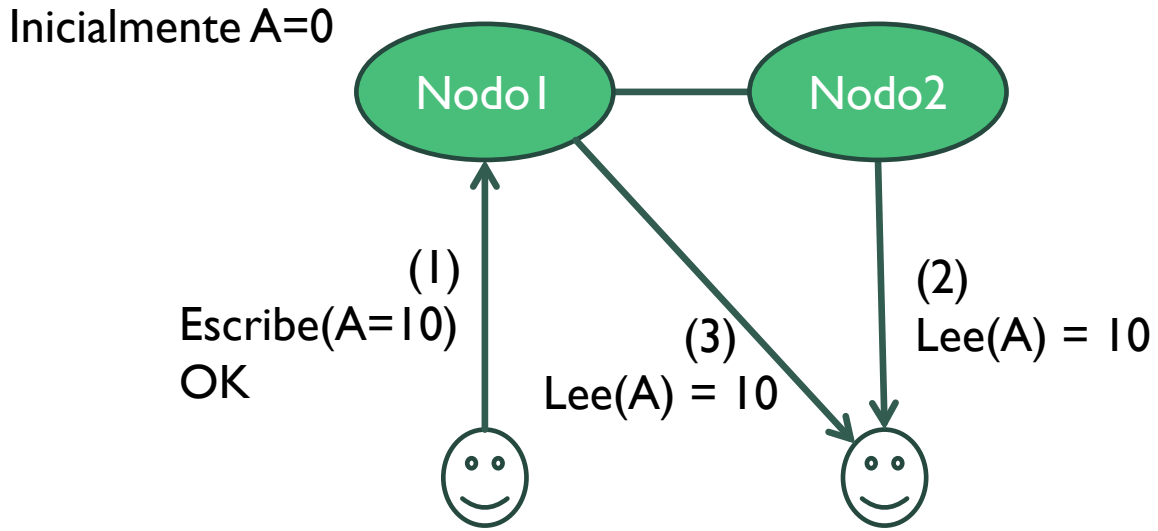
2. TEOREMA DE CAP



* Big Data Architect's Handbook

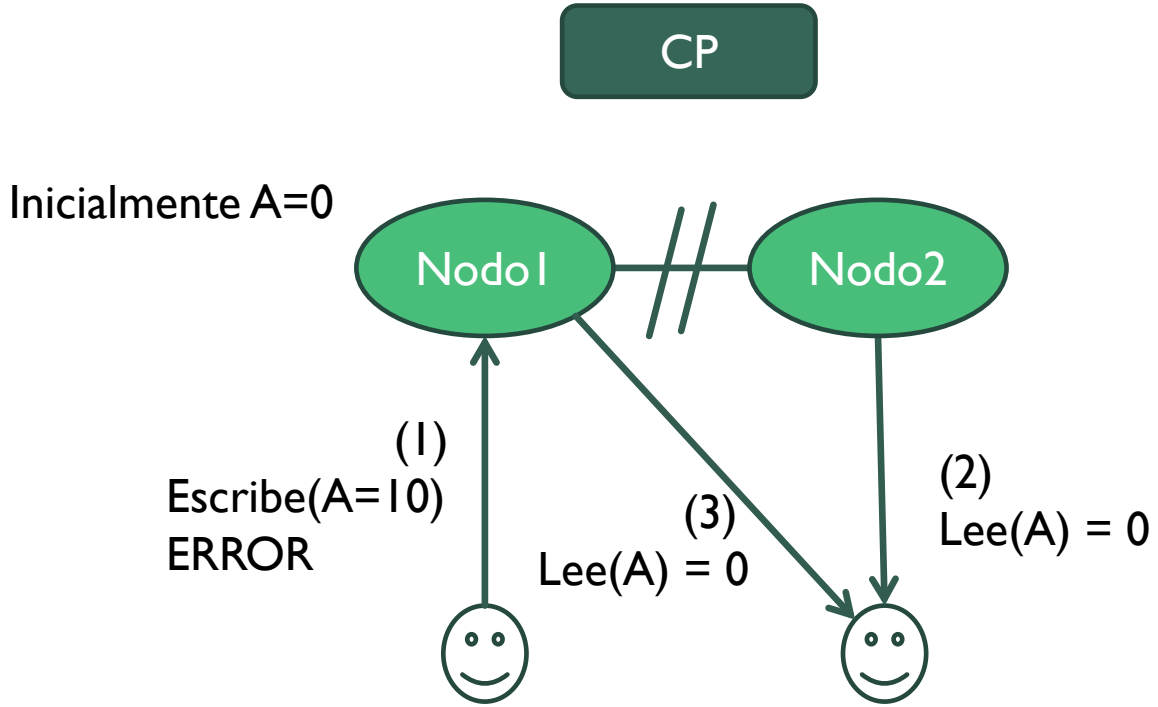
2. TEOREMA DE CAP

CA



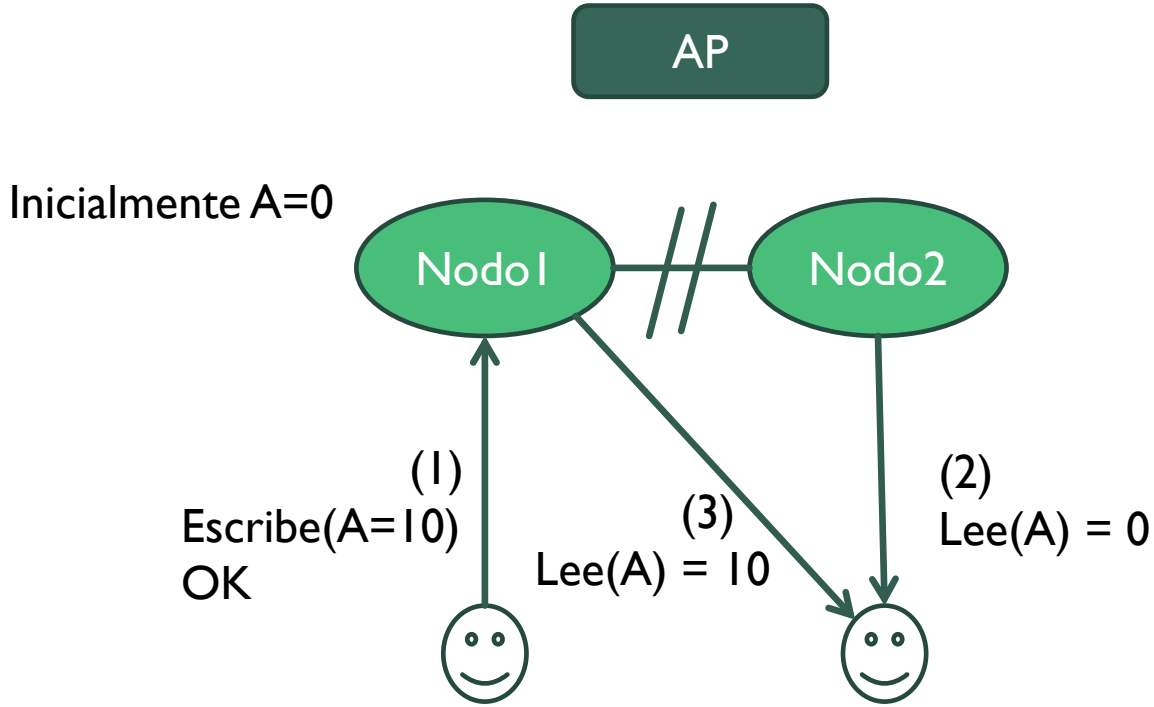
CA es propia de las bases de datos relacionales.

2. TEOREMA DE CAP



Nodo1 está activo pero no ha atendido una petición: No tiene **alta disponibilidad**.

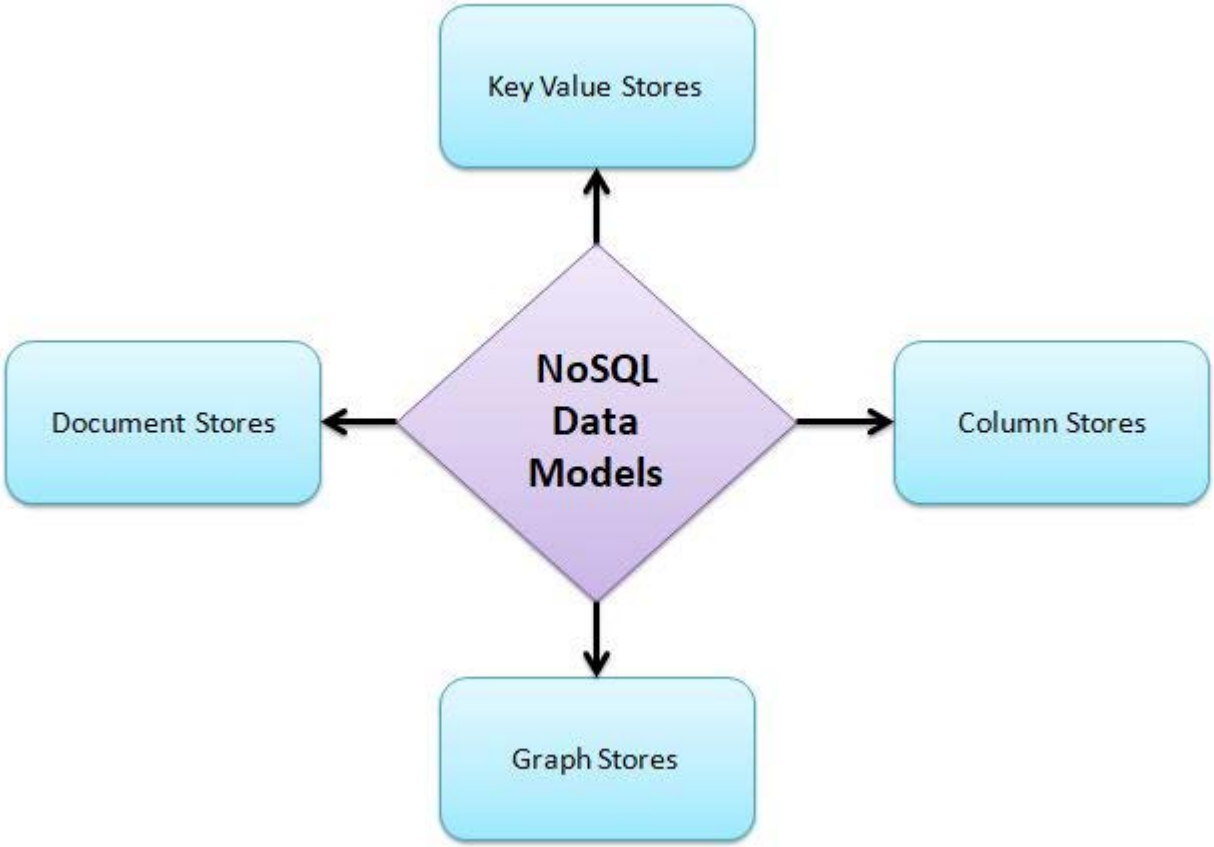
2. TEOREMA DE CAP



Cada nodo almacena un valor distinto para el dato A:
No hay **consistencia**.

3. MODELO DE DATOS

3. MODELO DE DATOS



* Big Data Architect's Handbook

3. MODELO DE DATOS: CLAVE-VALOR

- Almacenan datos en la forma de un par clave-valor, donde la clave es un identificador único y el valor contiene el dato que se desea almacenar.
- El dato puede ser de una gran variedad de formatos, como por ejemplo, JSON, vídeo, audio, … Por ejemplo, la clave *cust1237* puede apuntar al valor:

```
{name: "Douglas Adams",
 street: "782 Southwest St.",
 city: "Austin",
 state:  "TX"}
```

- Este tipo de bases de datos se utiliza normalmente para proporcionar alta disponibilidad y particionamiento de datos, que es la configuración AP del teorema de CAP.

3. MODELO DE DATOS: CLAVE-VALOR

- Características principales:
  - Consultas sencillas.
  - Modelado de datos flexible.
  - Altas prestaciones y alta disponibilidad.
  - Escalabilidad.
  - Portabilidad y bajos costes operacionales.
- Adecuada para:
  - Información de sesión en aplicaciones web.
  - Carritos de la compra.
- No adecuada cuando:
  - Se necesitan búsquedas en los valores.
  - Los datos se encuentran interconectados.



DynamoDB



3. MODELO DE DATOS: DOCUMENTOS

- Almacenan datos semi-estructurados, como JSON, XML, BSON.
  - Cada documento puede tener una estructura diferente al resto.
- Es similar al clave-valor ya que genera una clave única para cada fila para asociarla al documento.
- A diferencia de clave-valor, se pueden realizar búsquedas basadas en el valor.

3. MODELO DE DATOS: DOCUMENTOS

- Características principales:
  - Modelado de datos flexible.
  - Escrituras rápidas.
  - Lecturas rápidas.
- Adecuada para:
  - Almacenamiento de contenidos (CMS).
  - Blogging.
  - …
- Inadecuada para:
  - Aplicaciones que requieran consultas complejas.







3. MODELO DE DATOS: COLUMNAS

- Almacenan datos de forma tabular, pero organizando los datos en forma de columnas.
- Es altamente escalable y de muy altas prestaciones.
- Ejemplos de bases de datos: Cassandra, HBase, Hypertable, Amazon SimpleDB.







HYPERTABLE

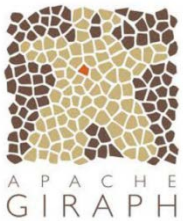
3. MODELO DE DATOS: COLUMNAS

- Características principales:
  - Consultas de **agregación** muy rápidas en comparación con bases de datos orientadas a filas.
  - Mejor compresión.
  - Si se necesitan múltiples columnas de cada fila, sus prestaciones caen.
  - Las escrituras requieren más tiempo ya que hay que escribir cada columna independientemente.

3. MODELO DE DATOS: GRAFOS

- Proporcionan una representación eficiente de los datos donde los registros tienen relaciones entre ellos.
- Aprovechan algoritmos optimizados para buscar en estructuras de datos basados en grafos.
  - Estas bases de datos pueden localizar información de forma diferente a otras propuestas NoSQL.
- En estas estructuras, de datos la información y sus propiedades se representan como aristas y nodos.







3. MODELO DE DATOS: GRAFOS

- Características principales:
  - Altas prestaciones en datos altamente relacionados.
  - Flexibilidad.
- Adecuadas para:
  - Recomendadores.

3. MODELO DE DATOS

- Diferencias entre bases de datos NoSQL:
  - Arquitectura:
    - Maestro-esclavo (ej. MongoDB) o *masterless* (ej. Cassandra).
  - Modelo de distribución de datos.
  - Modelo de desarrollo.
    - Complejidad de las consultas.

3. MODELO DE DATOS

- ¿Qué base de datos es más adecuada para cada caso?
  - Datos altamente relacionados:
    - Grafo.
  - Consultas de agregación:
    - Columnas.
  - Consultas basadas en los valores:
    - Documental.
  - En otro caso:
    - Clave-valor.





mongoDB









DynamoDB



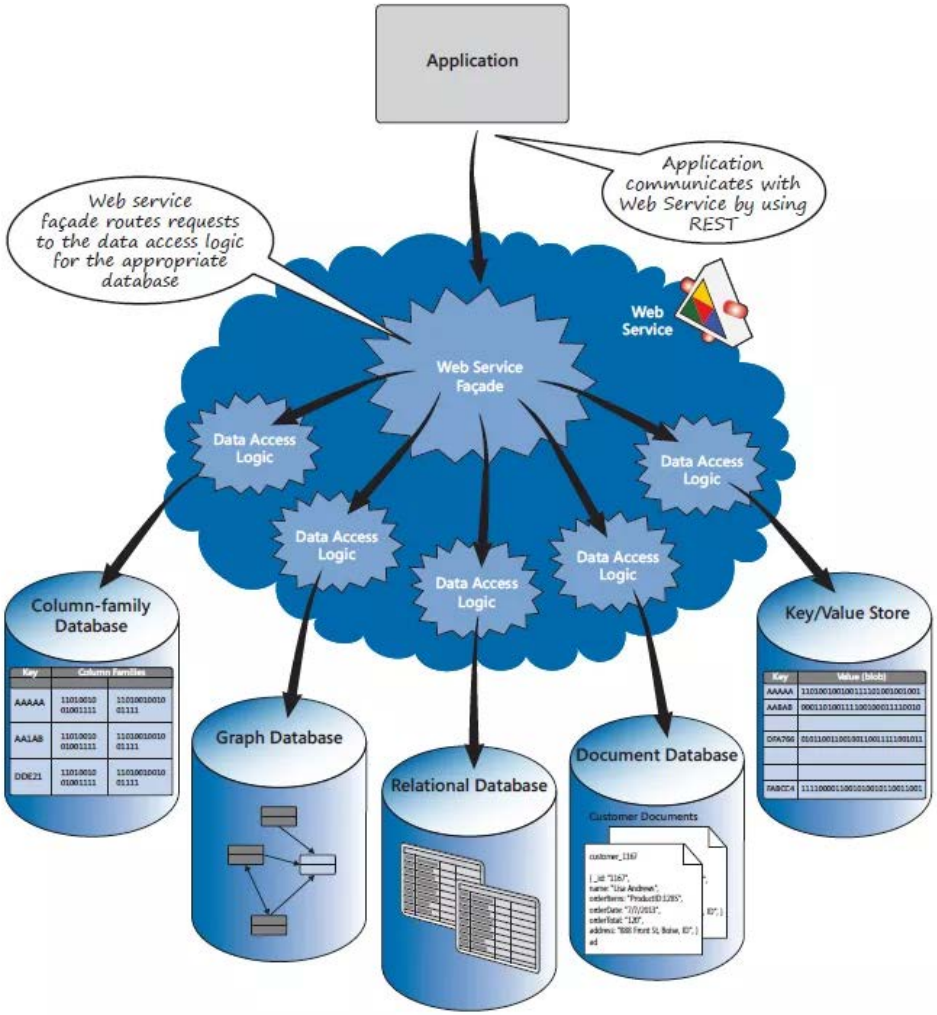
elasticsearch



cassandra

3. MODELO DE DATOS

- **Persistencia políglota:**
  - Utilización de la mejor herramienta para cada caso de uso en una aplicación concreta.
  - Ventajas:
    - Aprovecha las tecnologías subyacentes
  - Inconvenientes:
    - Desarrollo y mantenimiento del software más complejo.
  - Ejemplo: aplicación de comercio electrónico:
    - Carrito de la compra: NoSQL clave-valor.
    - Pagos: SQL.
    - Recomendación de productos: NoSQL grafos.
    - Pedidos: NoSQL documental.



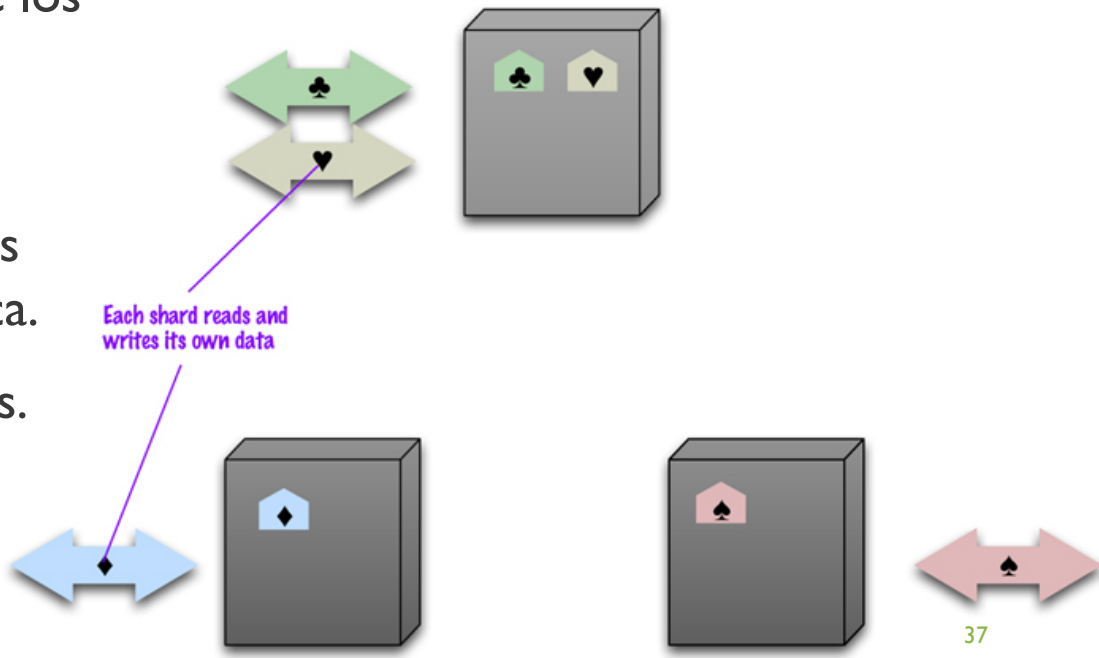
4. MODELOS DE DISTRIBUCIÓN

4. MODELOS DE DISTRIBUCIÓN

- Las tecnologías NoSQL están específicamente diseñadas para trabajar en un cluster, pero… ¿cómo gestionan los datos en el cluster?
  - Replicación.
    - Maestro-esclavo.
    - Peer-to-peer.
  - Sharding.

4. MODELOS DE DISTRIBUCIÓN

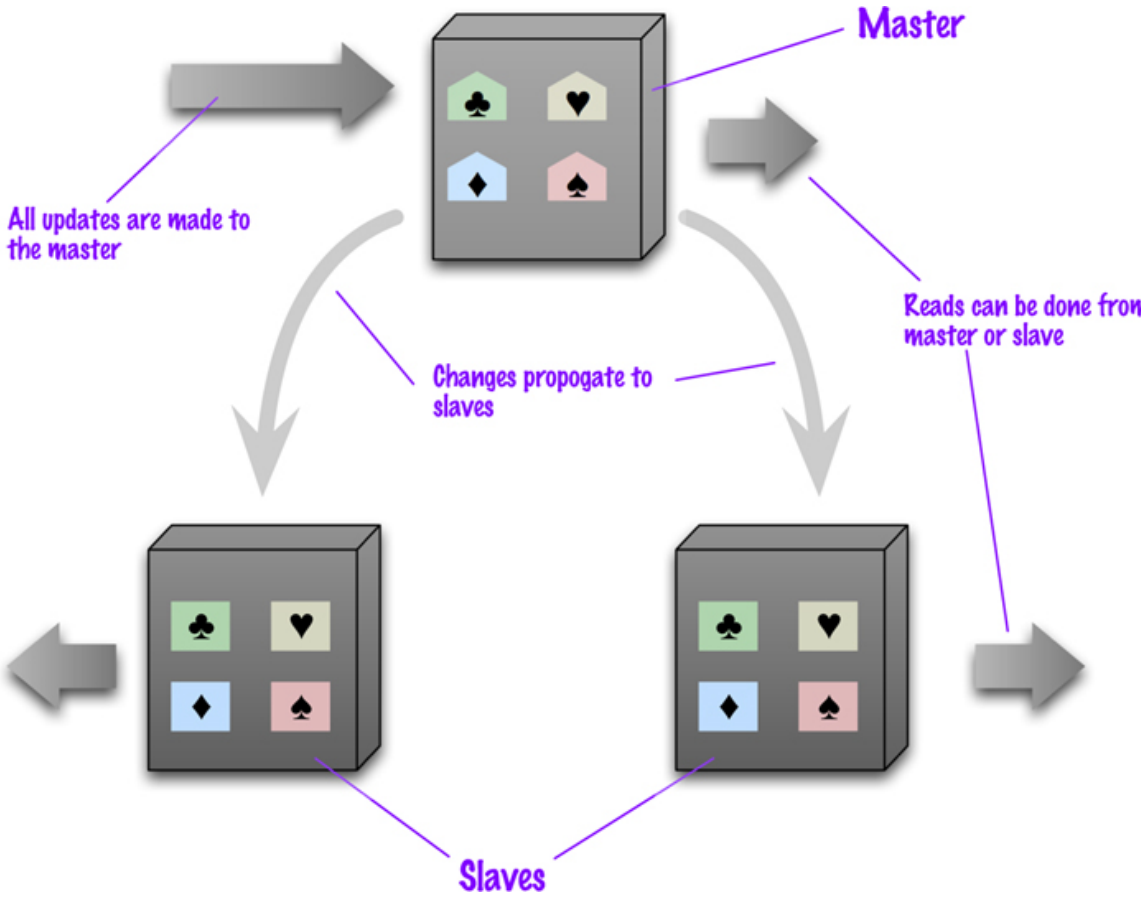
- **Sharding:**
  - Los datos se distribuyen entre los nodos del sistema.
  - Cada dato está en un nodo.
  - La asignación de datos a nodos puede ser manual o automática.
  - No mejora la tolerancia a fallos.



* NoSQL Distilled

4. MODELOS DE DISTRIBUCIÓN

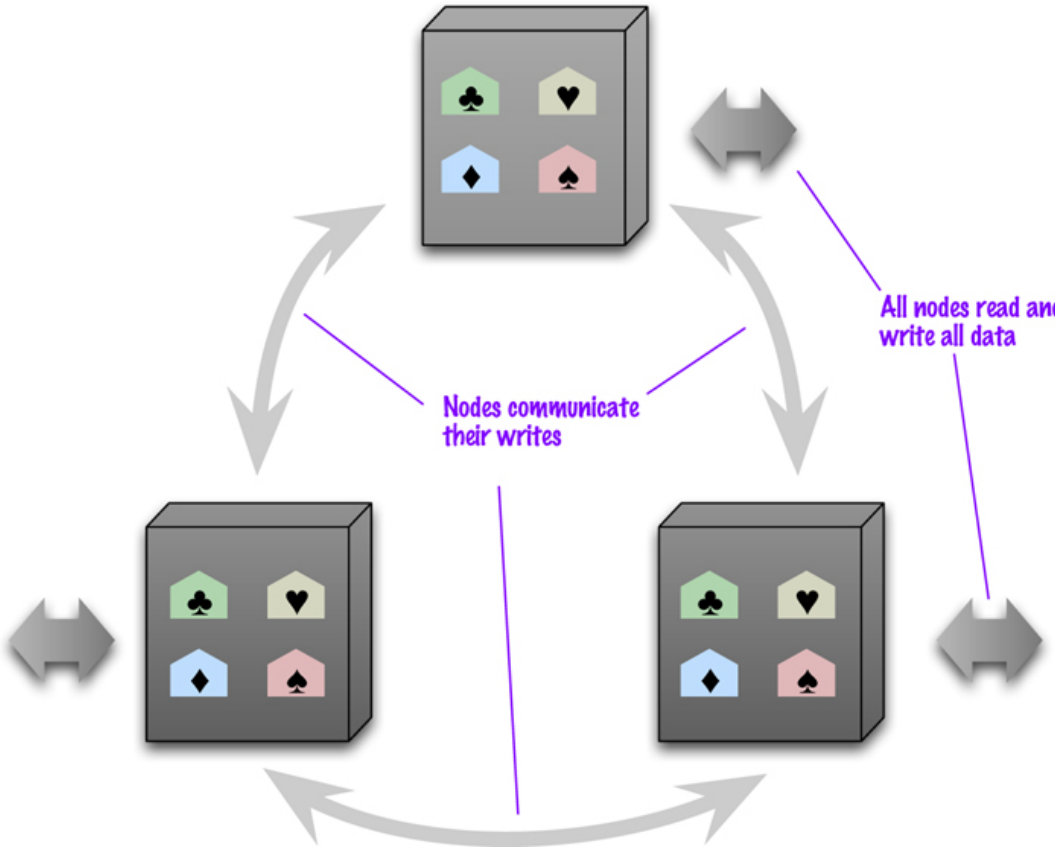
- **Replicación maestro-esclavo:**
  - Principalmente lecturas.
  - Lecturas tolerantes a fallos.
  - Problema: inconsistencia.



* NoSQL Distilled

4. MODELOS DE DISTRIBUCIÓN

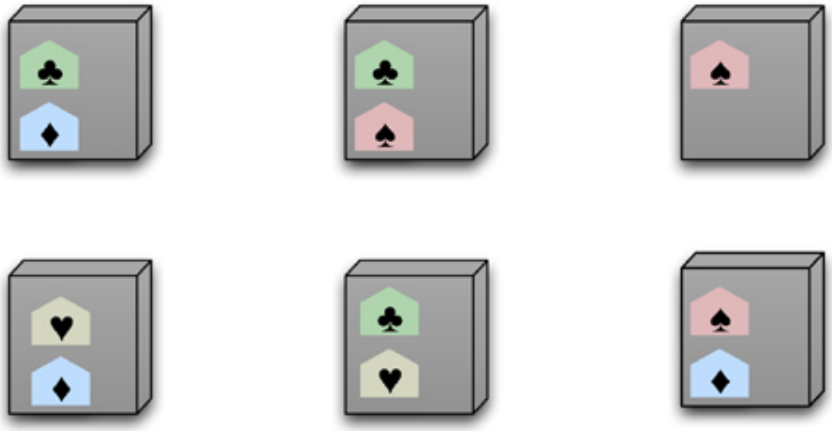
- **Replicación peer-to-peer:**
  - Todos los nodos leen y escriben todos los datos.
  - Tolerancia fallos de cualquier nodo.



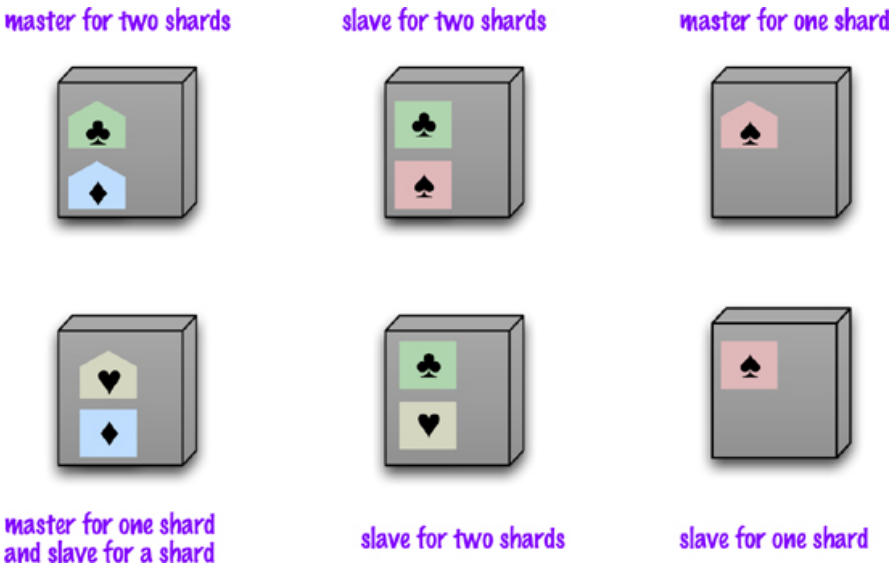
* NoSQL Distilled

4. MODELOS DE DISTRIBUCIÓN

- **Combinar replicación y sharding:**



Sharding y replicación P2P

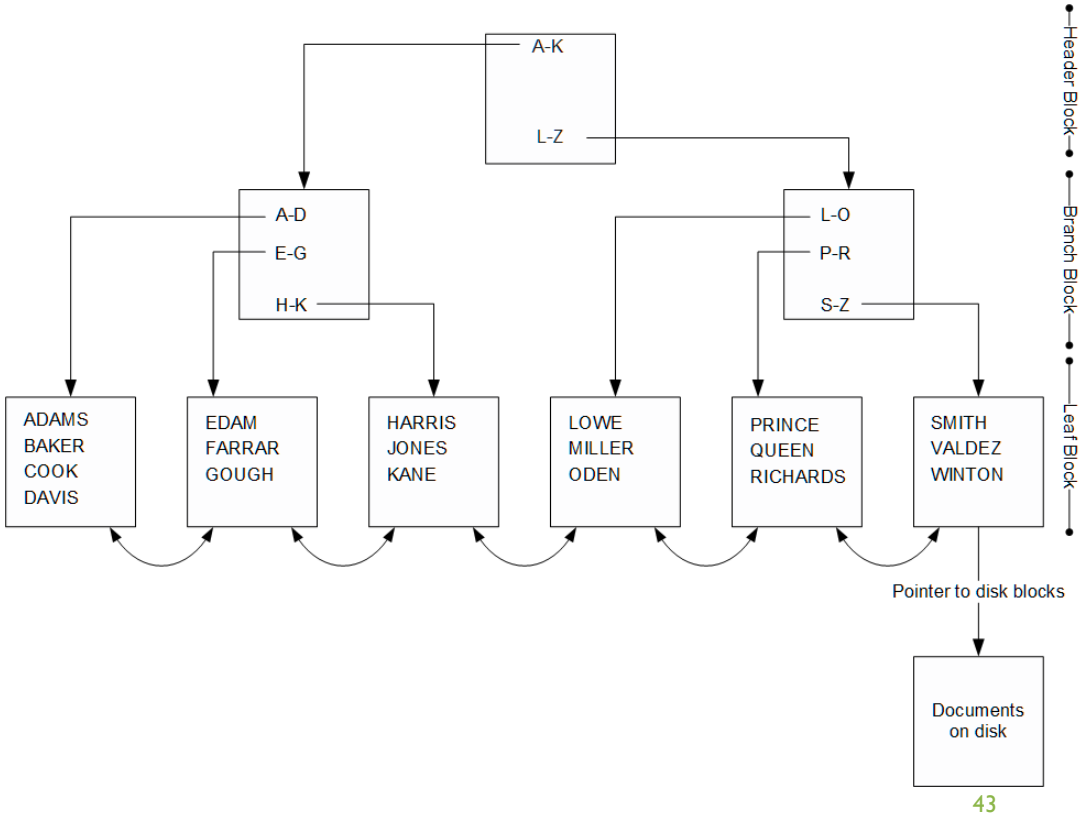


Sharding y replicación maestro/esclavo

5. INDEXACIÓN

5. INDEXACIÓN

- Los índices en una base de datos mejoran el acceso de lectura a los datos, a expensas de empeorar en las escrituras.
- Las estructuras de datos que se utilizan en los índices en bases de datos NoSQL son similares a las utilizadas en bases de datos relacionales.
  - B-tree



UNED

Departamento de Sistemas de Comunicación y Control

Tema 7: Fundamentos de bases de datos NoSQL

¿Preguntas?

Agustín C. Caminero Herráez
accaminero@scc.uned.es // @accaminero